

DESAFIO PRÁTICO | LISTAS E TUPLOS

Índices, alteração, cópia e processamento

Tempo sugerido: 40 minutos | Sem executar código | Justifica as respostas

Nome: _____ Turma: _____ Data: _____

1 ÍNDICES E FATIAS

```
v = [10, 20, 30, 40, 50]
```

Indica o resultado de cada expressão:

v[0]	_____
v[-1]	_____
v[1:4]	_____
v[:2]	_____
len(v)	_____

O que acontece ao avaliar v[5]?

2 ALTERAÇÕES NA LISTA

```
v = [10, 20, 30]
v.append(40)
v.insert(1, 15)
v.remove(20)
del v[0]
```

a) Escreve a lista após cada alteração.

b) Qual é a diferença entre remove(20) e del v[0]?

3 ATRIBUIÇÃO OU CÓPIA

```
a = [10, 20]
b = a
c = a.copy()
b[0] = 99
c.append(30)
```

a) Indica os valores finais de a, b e c.

b) Explica por que alterar b também altera o conteúdo observado através de a.

4 TUPLOS E IMUTABILIDADE

```
ponto = (3, 5)
x, y = ponto
```

a) Indica x, y e ponto[-1].

b) Que erro causa ponto[0] = 7?

c) Cria um novo tuplo (7, 5), mantendo ponto inalterado.

d) Como se escreve um tuplo com apenas o valor 10?

5 ORDENAR E SELECIONAR

```
v = [7, 2, 5, 2]
a = sorted(v)
b = [n for n in v if n > 3]
```

a) Indica v, a e b após o código.

b) Explica o problema de escrever v = v.sort().

6 DESAFIO INTEGRADOR

Dada uma lista notas com valores válidos entre 0 e 20, calcula a média, o mínimo, o máximo e uma nova lista de aprovados (nota ≥ 10). Se estiver vazia, mostra **Sem notas**.

a) Escreve o código.

b) Testa com [8, 12, 15, 7, 18] e com [].

DESAFIO PRÁTICO | LISTAS E TUPLOS

Índices, alteração, cópia e processamento

SOLUÇÕES

Compara os resultados e verifica o raciocínio.

1 ÍNDICES E FATIAS

Resultados:

```
v[0]    -> 10
v[-1]   -> 50
v[1:4]  -> [20, 30, 40]
v[::2]  -> [10, 30, 50]
len(v)  -> 5
```

v[5] provoca **IndexError**. Os índices não negativos válidos são 0 a 4.

Numa fatia, o índice final fica excluído.

2 ALTERAÇÕES NA LISTA

```
[10, 20, 30, 40]
[10, 15, 20, 30, 40]
[10, 15, 30, 40]
[15, 30, 40]
```

remove(20) remove a primeira ocorrência do valor 20. Se esse valor não existir, ocorre **ValueError**.

del v[0] elimina o elemento que está no índice 0, independentemente do seu valor.

3 ATRIBUIÇÃO OU CÓPIA

```
a: [99, 20]
b: [99, 20]
c: [10, 20, 30]
```

a e b referem-se à **mesma lista**. A atribuição b = a não copia os elementos.

a.copy() cria outra lista. Neste exemplo, os elementos são inteiros e as alterações feitas na lista c não afetam a.

4 TUPLOS E IMUTABILIDADE

a) x = 3, y = 5 e ponto[-1] = 5.

b) Ocorre **TypeError**: não se pode substituir um elemento de um tuplo.

```
novo = (7, ponto[1])
```

c) novo vale (7, 5) e ponto continua a valer (3, 5).

d) Escreve-se **(10,)**. A vírgula cria o tuplo com um único elemento.

5 ORDENAR E SELECIONAR

```
v: [7, 2, 5, 2]
a: [2, 2, 5, 7]
b: [7, 5]
```

sorted(v) devolve uma nova lista ordenada. A compreensão seleciona os valores superiores a 3.

b) v.sort() ordena a lista no próprio objeto e devolve **None**. Atribuir esse resultado a v faz v passar a referir-se a None.

Para ordenar no próprio objeto, basta usar **v.sort()**.

6 DESAFIO INTEGRADOR

```
if notas:
    media = sum(notas) / len(notas)
    resumo = (media, min(notas),
              max(notas))
    aprovados = [n for n in notas
                  if n >= 10]
    print(resumo)
    print(aprovados)
else:
    print('Sem notas')
```

Teste: resumo = (12.0, 7, 18); aprovados = [12, 15, 18]. Para [], mostra Sem notas, evitando divisão por zero e min/max de uma sequência vazia.